environment, you might want to lower those values to optimize resource consumption, since it increases linearly with the number of threads.

Listing 3.3 Configuring the Tomcat thread pool

```
server:
  port: 9001
  tomcat:
    connection-timeout: 2s
    keep-alive-timeout: 15s
    threads:
      max: 50
      min-spare: 5
```

So far, you've seen that cloud native applications with Spring Boot are packaged as JAR files and rely on an embedded server to remove extra dependencies on the execution environment and enable agility. You learned how the thread-per-request model works, familiarized yourself with the request processing flow with Tomcat and Spring MVC, and configured Tomcat. In the next section, we'll move on to the business logic of Catalog Service and the implementation of a REST API with Spring MVC.

3.3 Building a RESTful application with Spring MVC

If you're building cloud native applications, chances are that you are working on a distributed system composed of several services, such as microservices, interacting with each other to accomplish the overall functionality of the product. Your application might be consumed by a service developed by another team in your organization, or you might be exposing its functionality to third parties. Either way, there's an essential element in any inter-service communication: the API.